# Certifying quantum error-correction decoders: independently checkable near-optimality receipts at scale

Josh Philbrick[1, *]

[1] *Coherence Energy Labs*

(Dated: November 7, 2026)

A quantum error-correction decoder returns a correction and nothing else, and syndrome checking cannot detect a valid correction heavier than necessary. For minimum-weight decoding the correction is a minimum-weight $T$-join, whose linear-programming dual is a packing of $T$-odd cuts; any packing, scaled to feasibility, lower-bounds every correction by weak duality, whoever produced it. We build a producer of such packings and a checker that decides in exact rational arithmetic whether a packing proves its correction within $10^{-6}$ of optimal, and keep every receipt. On 1,000,000 simulated shots of a distance-5 surface code over 25 rounds, 99.81% of corrections were proven. On Google Willow shots decoded by PyMatching on the published error models, 99.60% of 99,384 were proven on ten configurations and 99.998% of 100,000 at distance 7 over 30 rounds, shots the producer was developed on; on held-out 50-round runs fixed in advance, all 10,000 at distance 3, 99.09% of 10,000 at distance 5 and all 800 completed at distance 7, with 100 more unfinished at a preregistered budget's stop. Every receipt replays under two Python verifiers, and a JavaScript checker written by a separate AI agent from this paper and a logged brief agrees with them on 48,969 replayed receipts and 89,984 fuzzed inputs. A sound checker refuses every correction heavier than the optimum by more than $10^{-6}$; what the experiments measure is the false-alarm rate, 1 of 770 unmodified corrections. Our producer takes a mean of 53 ms per shot (median 30 ms), about 3,200 times the mean cost of a batched decode; PyMatching's own sparse-blossom dual, exported by a patch, proves each of its 6,000 corrections exactly optimal for the integers it decodes with, but only 3 within $10^{-6}$ on the published weights; decoding, exporting and building that witness takes 5.9–6.9 times a plain decode per call, and 236–311 times with our exact check.

## I. INTRODUCTION

A decoder is software, and software has defects. An assurance argument for a fault-tolerant memory therefore has to answer a concrete question: if the decoder were subtly wrong, what would notice? In current practice the answer is syndrome consistency. A correction is accepted when the set of detectors it makes odd equals the set that fired, a property every functioning decoder satisfies by construction.

Syndrome consistency is blind to the failure that matters most for a minimum-weight decoder: a correction that explains the syndrome but is not the lightest one. A defective priority queue, a stale cache, or a mis-set tie-break can produce exactly such corrections. They pass every syndrome check, and detecting them by benchmarking means resolving a rate change against a noisy baseline, where decoder defects are confounded with device drift.

Minimum-weight decoding admits a better check. The problem is a minimum-weight $T$-join [1], its linear-programming dual is a packing of $T$-odd cuts, and weak duality makes any feasible packing a lower bound on every correction, regardless of the algorithm that produced it. A correction whose weight meets such a bound is optimal, and the packing is a witness that a third party can check in time proportional to its size and the edges it touches, without a solver and without trusting the decoder. This is in the spirit of the certifying algorithms of McConnell *et al.* [2] and the program checkers of Blum and Kannan [3], though not complete in their sense: our producer does not find a witness for every correct output.

This paper reports the engineering and measurement needed to make that observation carry weight in practice:

- a producer of packings whose receipts prove 99.81% of simulated corrections, and 99.60% (base ladder) and 99.998% (full ladder) of development hardware corrections, within $10^{-6}$ of optimal, with held-out samples fixed in advance (Secs. V and VI);
- an exact $\varepsilon$-checker, a second verifier in scaled-integer arithmetic, a third in JavaScript written by a separate AI agent from this paper and a logged brief, a differential fuzzer between them, and a forgery battery whose attacks are audited for falsity (Secs. III, IV and VI);
- a precise account of what a receipt proves when weights are IEEE-754 doubles, and a complete re-run of the scale experiments under it with every receipt kept (Secs. IV and VI);
- two detection experiments, on seeded decoder faults and on an upstream calibration fault, where syndrome checking detects nothing: their positive arm follows from weak duality, and what they measure is the false-alarm rate (Sec. VII);
- preregistered per-shot optimality-gap distributions for BP-OSD in two configurations and for belief-

* Josh@coherenceenergylabs.com

matching, with a post hoc $2 \times 2$ that separates BP-OSD's input model from its min-sum scaling (Sec. VIII); and

- the cost of certification, including what the decoder's own dual gives (Sec. IX).

Related work and what this paper does not claim are in Sec. X; the history of how the evidence was produced, including every defect found in it, is in Appendix A.

## II. CERTIFICATES FOR MINIMUM-WEIGHT DECODING

Let $G = (V, E, w)$ be a decoding graph with non-negative edge weights and a virtual boundary vertex $b$, and let $T \subseteq V$ be the set of fired detectors, with $b$ added when $|T|$ is odd so that $|T|$ is even. A correction is an edge set $J \subseteq E$ whose odd-degree vertices are exactly $T$, a $T$-join; minimum-weight decoding asks for one of least weight $w(J) = \sum_{e \in J} w_e$ [4, 5].

A set $S \subseteq V$ is $T$-odd when $|S \cap T|$ is odd, and $\delta(S)$ denotes the edges with exactly one endpoint in $S$. Every $T$-join meets every $T$-odd cut, which gives the packing problem

$$\max \sum_S z_S \quad \text{s.t.} \quad \sum_{S:\, e \in \delta(S)} z_S \le w_e \;\; \forall e \in E, \qquad z \ge 0, \tag{1}$$

over $T$-odd sets $S$. For any feasible $z$ and any $T$-join $J$,

$$w(J) \;\ge\; \sum_{e \in J} \sum_{S:\, e \in \delta(S)} z_S \;=\; \sum_S z_S \, |J \cap \delta(S)| \;\ge\; \sum_S z_S, \tag{2}$$

so a packing whose value equals $w(J)$ proves $J$ minimum. Two properties make this a useful certificate. The argument never refers to how $z$ was obtained, so the checker need not trust the producer. And the constraints are edge-local: feasibility is verified by accumulating, per edge, the values of the sets whose boundary it crosses.

Existence is not in question. Edmonds and Johnson showed that the $T$-cut constraints describe the dominant of the $T$-join polytope [1], so the linear program (1) attains the minimum $T$-join weight and an attaining packing exists for every syndrome. What the rates below measure is therefore how often our producer finds one, not whether one exists; Sec. VI C checks the decoder's side independently. Node potentials are a certificate only in one special case: non-negative values on the singletons $\{v\}$, $v \in T$, checked against every edge, are a packing (1) in which each edge carries the values of its endpoints in $T$. Potentials that may be negative, or that sit on vertices outside $T$, can satisfy $y_u + y_v \le w_e$ on every edge and still sum to more than the optimum; they are the dual of a different problem. The checkers here accept packings of $T$-odd sets only, and the exact checker and the $\varepsilon$-checker refuse nonzero node potentials (Appendix A).

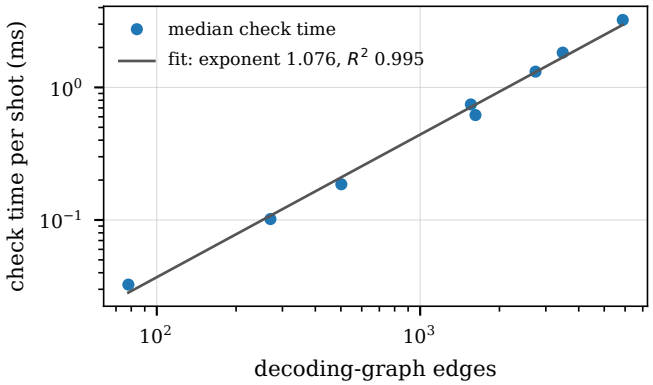



FIG. 1. Median time to check one certificate against the number of edges in the decoding graph, with a power-law fit, for the checker of the original runs (measured before the $\varepsilon$-checker existed; the $\varepsilon$-checker's cost at one size is in Sec. IX).

## III. THE CHECKER

Given the graph, the syndrome, a correction and a packing, the checker (i) verifies that the correction's odd-degree set equals $T$; (ii) verifies that every $z_S$ is non-negative and every $S$ is $T$-odd, since admitting an even set lets the bound exceed the optimum; (iii) accumulates the load every edge carries; and (iv) scales the packing by the largest factor, at most one, that makes every edge constraint hold and compares the scaled value with $w(J)$ in exact arithmetic, reporting the difference as the proven slack (Sec. IV). A value above $w(J)$ is impossible for valid input and is reported as a verifier failure, never accepted as a better bound. Loads are accumulated from each set's boundary, so the work is proportional to the total size of the listed sets and of the edges incident to them, not to the graph size times the number of sets; Fig. 1 shows median check time growing with exponent 1.076 in the number of edges ($R^2 = 0.995$).

*The input domain.* A checker is part of the trust boundary only if it decides exactly the documents it says it decides. All three implementations below enforce one stated contract: vertex identifiers are integers (the boundary is $-1$), never booleans or floating-point numbers; the graph is simple, so a self-loop or an edge listed twice in either orientation is refused (every decoding graph here is simple, checked on each persisted instance); every weight and packing value is a hexadecimal binary64 literal that names its value exactly, finite and non-negative; every packing set and correction endpoint is a vertex of the graph; and a set or a correction edge listed twice is refused, because a packing is a map from sets to values and a correction is a set of edges. Anything else is refused as invalid input, never coerced. The contract was made one policy on 2026-11-03, when a differential fuzzer showed the Python readers coercing inputs the third implementation refused (below).

*Certificates that travel.* A certificate is useful only if a stranger can re-check it. We call the record kept for

one shot a *receipt*: the fired detectors, the correction, the packing, and the checker's verdict with its proven slack and bound, every floating-point value written as an exact hexadecimal literal. A receipt is bound to its graph by a fingerprint the verifier recomputes from the graph it is given (for the reruns of Sec. VI, the hash of the instance file that each configuration's summary records), so a genuine certificate for an easy instance cannot be replayed against a hard one; and a shot that does not certify still receives a receipt, labelled not proven, with its slack, so that failure and silence are distinguishable. A signature attests authorship only.

*Second and third implementations.* A checker written by the same author in the same language cannot expose a shared misreading of the problem, and two checkers fed by one parser cannot expose that parser's. The results of Sec. VI rest on the $\varepsilon$-checker of Sec. IV; every receipt is re-decided by a second verifier written separately in scaled-integer arithmetic, in the same language. We therefore also wrote a third, `verifier-js/verify-epsilon.mjs`. A fresh AI agent wrote it from Secs. II–IV of this paper and a brief (`docs/reviews/JS_EPSILON_BRIEF_2026-11-03.md`) that gave the wire format and the kinds of malformed input the fuzzer below would send; the brief's author had read the Python checkers. The writing agent had access to the whole file system: it was told not to open the Python sources and reported that it did not, and the list of the files it opened, released with the brief, is our summary of its report, not a log. It is the same project's work, and what it offers is an independent reading, not independent authorship. A differential fuzzer (`tools/fuzz_epsilon_checkers.py`) compared the three on random valid receipts over small graphs, decided across $\varepsilon_{\max}$ in both directions, and on one-defect malformed mutants of each. On every valid input the three returned the same status, slack and bound. On malformed input they first split in 20 classes (34,894 cases), every one rooted in the Python readers and most shared by both Python verifiers: the receipt loader read 0.9 as vertex 0 and `true` as vertex 1, and read the decimal-looking literal `0.5` as hexadecimal 0.3125; the second verifier accepted a self-loop and `true` as a detector and raised, rather than refused, on a non-finite weight; the first collapsed a parallel edge to the lighter copy. None of these could accept a false claim about a well-formed receipt, but each read a malformed document as some other document, which is the shared-misreading failure the second implementation exists to catch and the same-language pair could not. After the fix all three agree on 89,984 fuzzed inputs (14,812 valid), every class is a standing conformance vector (153 vectors, 51 classes) run against all three, and the JavaScript checker returns the recorded verdict, slack and bound on 48,969 replayed receipts from 8 released configurations and Sec. VI E.

The earlier JavaScript checker, `verifier-js/verify-certificate.mjs`, implements the exact checker only, and it is less independent than first described. Its parser, graph binding, $T$-join test and $T$-oddness test (with $b$ counted in $T$, Sec. II) were written from the specification; its exact-arithmetic acceptance step was ported from the Python checker and kept identical to it, and the two shared at least one misreading of boundary edges for a month (Appendix A). It and the exact checker agree on all 59 conformance vectors, 51 of which are refusals; agreement on the ported step is not independent evidence.

## IV. WHAT A RECEIPT PROVES UNDER FLOATING-POINT WEIGHTS

Decoding weights are log-likelihood ratios stored as IEEE-754 doubles, and packings are computed in floating point. Exact equality between a packing's value and $w(J)$ is therefore rarely observable, and a checker must decide what to accept. The choice changes the claim, so we separate two claims:

- *exact*: $w(J) = \min_{J'} w(J')$ over the double-valued weights, exactly;
- *$\varepsilon$-optimal*: $w(J) - \min_{J'} w(J') \le \varepsilon$, with $\varepsilon$ proven.

Every double is a rational number, so either claim can be decided in exact arithmetic. The following lemma turns any non-negative packing over $T$-odd sets, feasible or slightly overloaded, into a proven slack.

**Lemma 1.** *Let $z \ge 0$ be supported on $T$-odd sets, let $\ell_e = \sum_{S:\, e\in\delta(S)} z_S$, and let $\theta = \min\bigl(1, \min_{e:\,\ell_e>0} w_e/\ell_e\bigr)$. Then $\theta z$ is feasible for (1), and for every $T$-join $J$ with minimum $J^\star$,*

$$w(J) - w(J^\star) \;\le\; \varepsilon(J,z) \;:=\; w(J) - \theta \sum_S z_S.$$

*Proof.* Each constraint of $\theta z$ reads $\theta\ell_e \le w_e$, which holds by the choice of $\theta$. Applying (2) to $\theta z$ and $J^\star$ gives $w(J^\star) \ge \theta \sum_S z_S$. □

*The checker the results use.* The $\varepsilon$-checker (`check_epsilon`) decides the $\varepsilon$-claim itself. It converts every weight and dual value to an exact rational; refuses input outside the domain of Sec. III; computes $\theta$ and $\varepsilon(J,z)$ exactly; and returns one of five outcomes: *exactly optimal* ($\varepsilon = 0$), *$\varepsilon$-optimal* ($0 < \varepsilon \le \varepsilon_{\max}$, with $\varepsilon$ reported), *not proven* ($\varepsilon > \varepsilon_{\max}$), *invalid input*, or *verifier failure*, a bound above the correction's weight, impossible for valid input and never an acceptance. A refusal is not evidence that the decoder erred; only a lighter valid correction is. The headline rates of Sec. VI are acceptances of this checker with $\varepsilon_{\max} = 10^{-6}$, from complete reruns that kept every receipt, and every receipt is re-decided by the second verifier, which reads the receipt's lists itself, and the replay stops on the first receipt where the two, or either and the recorded verdict, disagree. The checker's tests kill 106 of 109 mutants generated mechanically by standard mutation

operators; each of the remaining 3 computes the same function as the original, for a reason recorded with the score.

*The rule the original runs used.* The runs first made for this work accepted a certificate when its floating-point gap was at most $10^{-6}$ and no edge was overloaded by more than $10^{-9}$. An external audit of 2026-08-04 showed that such a rule is not a proof: it accepts a correspondingly small suboptimality (a correction of weight 1.0000005 was accepted against a true optimum of 1), it had no guard against non-finite values, and its checker never tested that a packing's sets are $T$-odd (Appendix A). Those runs did not keep their certificates, and their rates are kept in Sec. VI only as a development record, labelled as tolerance-rule acceptances wherever they appear. To measure what the rule actually certified, we re-ran the exact code that produced each headline artifact, extracted from its commit, on seeded samples of six configurations, kept every certificate it accepted, and analysed each in exact rational arithmetic (Table I). None of the 1,588 accepted certificates was inadmissible, and the largest proven slack was $2.6 \times 10^{-13}$. For the runs it sampled, that bounds the inadmissible share of their acceptances below 0.75% for the simulation, 2.00% for Willow at distance 5 over 13 rounds (X) and 7.22% at distance 7 over 30 rounds (X), one-sided 95% each; no other configuration or generation was sampled.

*Exact certification is a different and harder claim.* The exact checker, `matching_cert.check`, accepts only a proof of exact optimality: it repairs the floating-point rounding of a packing in exact arithmetic and then verifies feasibility and attainment from scratch. It accepts 842 of the 1,588 certificates above (53%), and the fraction falls steeply with graph size (Table I). Part of the refusals are the decoder's own: PyMatching matches on internally discretised weights, and among the 770 unmodified corrections of Sec. VII, 14 exceeded the exact optimum over the double-valued weights by a positive amount of at most $10^{-6}$. Because a decoder that computes in floating point is not attempting exact optimality over double-valued weights, and because those weights are estimates far coarser than $10^{-13}$, we regard $\varepsilon$-optimality with a small, proven $\varepsilon$ as the operationally meaningful claim, and report exact certification separately.

## V. PRODUCING THE WITNESS

Producing a packing is the expensive half. *Moat growth* grows every $T$-odd component at equal rate and merges components when an edge becomes tight, as in the primal–dual approximation of Goemans and Williamson [6]; unlike Edmonds' exact blossom algorithm [7], it does not revise its choices. It is feasible by construction and 17–64 times faster than solving a linear program, but at distance 7 it certifies 31/120 shots against 113/120 for the linear program, on the same shots (tolerance rule, development), because greedy merging picks the wrong pairs.

The main producer solves (1) restricted to a candidate family of sets. Restricting the family cannot cost soundness, since a feasible point of the restricted problem is feasible for the full one; it costs only tightness, which appears as a reported gap. The groups worth adding are read from the packing's own slack: a set can carry more value only if every edge on its boundary has slack, so the connected components of the subgraph of tight edges are added and the problem solved again. The producer is a ladder whose rungs are tried cheapest first, escalating only on shots the previous rung left open: tight-component refinement; sets that cut the correction exactly once, as complementary slackness requires of an optimal dual; candidates translated from the duals of a cutting-plane linear program for perfect matching on the metric closure over $T$; an escalation tier that raises every binding budget on refused shots; and a final exact tier that generates $T$-odd cuts by Padberg–Rao separation [8], capped at a round budget. Why the translated candidates leave some shots open for the later tiers is not determined: we have not measured which of the cutting-plane program's non-convergence, the quantisation of its values, the per-detector boundary copies of the metric closure, or the candidate budget dominates on the shots they leave open. The correction never changes between rungs, and every rung is judged by the same checker. We call the rungs up to the closure candidates the *base ladder*, and the base ladder followed by the escalation and exact tiers the *full ladder*. None of these rungs reads the dual the decoder itself computed; Sec. IX measures what that dual gives.

## VI. RESULTS

Every printed figure errs against the claim it supports: a proven fraction is rounded down, a refusal rate or an upper bound up, and an interval's endpoints outward; times, weights and $p$-values are rounded to nearest unless stated as a bound. An interval is the exact two-sided 95% Clopper–Pearson interval; a bound stated alone is the exact one-sided 95% Clopper–Pearson upper bound. A *configuration* is a distance, a number of rounds and a basis.

### A. Simulation at scale

We sampled 1,000,000 shots of a distance-5 rotated surface-code memory over 25 rounds at circuit-level depolarising noise $p = 0.005$, generated with Stim [9] and decoded with PyMatching [5] on a graph of 600 detectors plus the boundary and 2,742 edges, following the seed plan of the original run chunk for chunk. Each correction went through the base ladder and was judged by the $\varepsilon$-checker, every chunk completed (0 retries), and every receipt was kept. Of 1,000,000 nontrivial corrections,

TABLE I. Exact re-analysis of the certificates accepted by the code that produced each reported artifact, named by its commit in the project's development repository; the source of these four commits ships as release tarballs (Sec. XI). "Largest proven slack" is $\max \varepsilon(J, z)$ from the lemma, computed in rational arithmetic. The last two columns show how the exact checker, which demands exact optimality, treats the same certificates, and how many of the same shots the current producer certifies under that stricter rule, using its base ladder and escalation tier but not its exact separation tier.

| configuration (code commit) | sample | accepted then | largest proven slack $\varepsilon$ | exact checker, same certificate | current producer (with escalation) |
|---|---|---|---|---|---|
| simulation, $d = 5$, 25 rounds (`d4a81ee`) | 400 | 399 | $1.4 \times 10^{-13}$ | 54 | 151 |
| simulation, $d = 3$, 3 rounds (`cce1e91`) | 230 | 230 | $2.7 \times 10^{-15}$ | 213 | 228 |
| simulation, $d = 5$, 5 rounds (`cce1e91`) | 395 | 392 | $1.9 \times 10^{-14}$ | 339 | 388 |
| simulation, $d = 7$, 7 rounds (`cce1e91`) | 400 | 378 | $7.2 \times 10^{-14}$ | 124 | 292 |
| Willow, $d = 5$, 13 rounds, X (`06344ae`) | 150 | 149 | $4.7 \times 10^{-14}$ | 110 | 135 |
| Willow, $d = 7$, 30 rounds, X (`f57e87d`) | 40 | 40 | $2.6 \times 10^{-13}$ | 2 | 3 |

998,145 (99.81%) were proven within $\varepsilon_{\max} = 10^{-6}$ of optimal, with a largest proven slack of $6.2 \times 10^{-12}$, and 8,628 of them with $\varepsilon = 0$ on the packing as produced (we write "with $\varepsilon = 0$" for this outcome of the $\varepsilon$-checker, which repairs nothing; exact certification, with repair, is the different claim of Sec. IV); the remaining 1,855 (0.19%) were not proven. On 10,000 further shots of the same configuration from seeds far from every recorded seed base, drawn after the first round of AI-agent review of this manuscript, 9,981 (99.81%; 99.70–99.89%) were proven and 19 were not (post hoc). The same experiment under the tolerance rule accepted 997,594 of 1,000,000 corrections (99.75%), from chunks with memory failures replaced by supplements (Appendix A); the rerun has no such gap.

*The forgery battery, and what its zero meant.* For every shot with a packing we built forged certificates from the strongest packing found: corrections with a dropped, duplicated or mismatched edge; the correction that sends every fired detector to the boundary, kept when it is heavier than the decoder's by more than $\varepsilon_{\max}$ (the cheapest-boundary class); three "strictly heavier" corrections meant to reach the bound (a spurious cycle, the packing inflated to meet the heavier weight, and the same with a negative value hiding the inflation); and the genuine packing with one value replaced by NaN or by $+\infty$. In the scale reruns the three stealth classes did not do what they were for: each listed one edge twice, which as an edge set is the genuine correction, and the $\varepsilon$-checker refuses a repeated edge as malformed input before computing any bound. Of the 8,000,001 forgeries on the simulation, 3,000,000 failed the syndrome or repeated-edge test, 3,000,000 were these doubled-edge documents, 2,000,000 carried a non-finite value, and the remaining 1, a cheapest-boundary correction, was the only one that reached weak duality. That class exists only on a shot whose every fired detector has an edge to the boundary: 312 of the simulation graph's 600 detectors have one, and the class arose on 1 of its 1,000,000 shots, while on the four distance-3 Willow graphs, where every detector has one, it arose on at least 88% of the shots of each configuration. A forgery total therefore says little by itself, and each total below is given with the number of its forgeries that only the bound could refuse.

*The rebuilt battery.* The battery now adds an even cycle disjoint from the correction, a valid and strictly heavier edge set; keeps the doubled-edge documents under their true name; and its audit refuses an attack whose correction lists an edge twice. Across one accepted receipt of each chunk of the 18 complete released configurations in the strict replay (59,224 forgeries, 10,034 of them refused by the bound), every receipt of Sec. VI E (462,794, 122,767 by the bound) and every receipt of the IBM run (21,169, 4,748 by the bound), no forgery was accepted. It includes the linear-programming adversary that found the one real checker defect, packings over sets that are not $T$-odd once $b$ is counted in $T$ (Appendix A): of its 4,920 forgeries in the strict replay, copies of both verifiers with only their parity test removed accepted 4,920, so every one was a real attack, and both real verifiers refused every one as invalid input. The replay also attacks the acceptance boundary, with one correction edge's weight raised just past or just inside $\varepsilon_{\max}$, and replays receipts against another configuration's graph, where none may be accepted (the procedures are in Appendix A). These counts show that the attacks tried were refused; that no forgery can succeed is what the lemma says.

### B. Google Willow

We applied the producer to detector samples from Google's Willow processor [10, 11]. The corrections are PyMatching's, computed on the detector error models the experimenters published with the data, so the weights are not ours to choose; the decoders Google used are not what is certified. The producer was developed in generations, each measured under the tolerance rule (Table II), each introduced against the shots its predecessor left open. Generation 1 accepted 78.44% of 596,732 nontrivial shots; generations 2 and 3 ran partial samples, down to 800 shots for one configuration; generation 4 ran every configuration at 10,000 shots.

TABLE II. Percentage of Willow shots accepted under the tolerance rule of the original runs, by producer generation, rounded down; each row holds two configurations, the X / Z basis. Generation 1 ran 50,000 shots per configuration; generations 2 and 3 ran partial coverage; generation 4 ran 10,000 per configuration.

| distance, rounds | gen. 1 | gen. 2 | gen. 3 | gen. 4 |
|---|---|---|---|---|
| $d = 3$, 13 rounds | 96.19 / 96.64 | 99.15 / 99.39 | 99.98 / 99.97 | 100.00 / 100.00 |
| $d = 3$, 30 rounds | 89.62 / 89.84 | 98.78 / 98.96 | 99.97 / 99.96 | 100.00 / 99.98 |
| $d = 5$, 13 rounds | 88.21 / 89.30 | 95.86 / 96.36 | 98.71 / 98.91 | 99.67 / 99.71 |
| $d = 5$, 30 rounds | 67.67 / 69.67 | 89.40 / 89.49 | 95.86 / 95.85 | 98.75 / 98.96 |
| $d = 7$, 13 rounds | 78.46 / 78.53 | 90.67 / 90.57 | 95.40 / 95.63 | 98.30 / 98.49 |
| $d = 7$, 30 rounds | 49.02 / 49.30 | 73.62 / 76.48 | 84.80 / 86.45 | 93.76 / 94.71 |
| aggregate | 78.44 | 93.08 | 96.03 | 98.52 |
| nontrivial shots | 596,732 | 107,184 | 117,235 | 119,384 |
| per configuration | 47,631–50,000 | 800–10,000 | 9,300–10,000 | 9,560–10,000 |

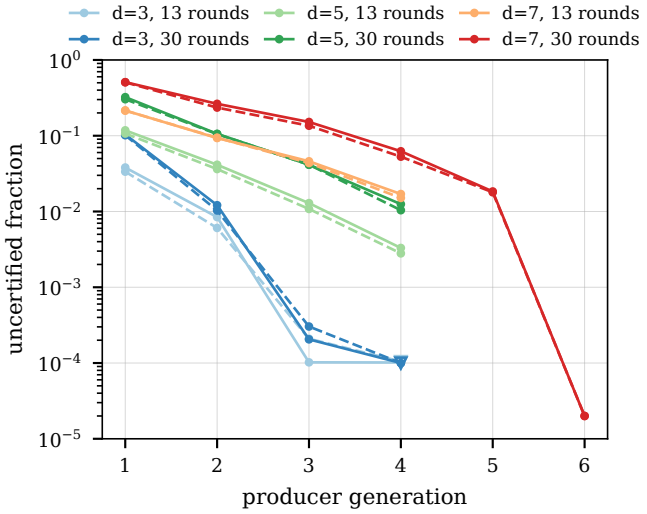



FIG. 2. Fraction of Willow shots not accepted under the tolerance rule, per configuration and producer generation; solid lines X basis, dashed Z. Generations 5 and 6 re-ran only distance 7 over 30 rounds. Where no shot was left uncertified, the point is drawn at $1/n$.

The two hardest configurations, distance 7 over 30 rounds in either basis, were re-run at 50,000 shots each, first with the escalation tier (98.16% and 98.20%) and then with the exact tier added, which left one shot per basis open (99.998%), at gaps 0.80 and 0.11; an exact-arithmetic oracle finds PyMatching's correction exactly optimal on both, so both refusals were producer slack at a budget. Counting each configuration at its final generation, 198,770 of 199,384 hardware shots were accepted under the tolerance rule (99.69%). Fig. 2 shows the uncertified fraction per configuration across generations.

*The rerun.* We reran every configuration at the generation the count above takes it from, on the same shots, with the $\varepsilon$-checker and every receipt kept (Table III). Of 199,384 nontrivial corrections, 198,987 (99.80%) were proven within $\varepsilon_{\max}$ of optimal: 99.60% on the ten configurations run with the base ladder and 99.998% at distance 7 over 30 rounds with the full ladder, which accounts for 50% of the shots. Of all of them, 44,157 were proven with $\varepsilon = 0$ and 397 were not proven; the largest proven slack was $1.4 \times 10^{-11}$, and none of 1,629,828 forgeries was accepted, 37,218 of them refused by the bound (the battery before it was rebuilt).

*Held-out samples.* These rates are measured on the shots the producer was developed on. To measure the producer on data it never saw, we fixed two protocols in commits of the project's development repository dated before any of their shots was certified. That repository is private and its history is not public, and a commit's date is set by whoever makes it, so this order rests on trust in the author; Sec. XI says what a reader can check instead. The first (`docs/PREREG_heldout_willow_2026-11-02.md`) froze the base ladder and certified the first 5,000 shots of four configurations at 50 rounds, distance 3 and 5 in both bases, which no generation had read, on the same two patches of the chip as the development data: the archive's sub-grids `d3_at_q4_5` and `d5_at_q8_7`, one per distance. Of 20,000 nontrivial corrections, 19,909 (99.54%; 99.44–99.64%) were proven within $\varepsilon_{\max}$ of optimal and none of 169,999 forgeries was accepted, 9,999 of them refused by the bound: all 10,000 at distance 3 and 9,909 of 10,000 (99.09%) at distance 5, against 99.76% and 99.24% for the same producer on the development shots at distance 5 over 13 and 30 rounds, which left 46 of 19,998 and 152 of 20,000 corrections not proven (Table III's earlier producer left 46 and 151; the one verdict that differs is described below).

The held-out runs differ from the development runs in their number of rounds as well as their shots, and at distance 5 the rate falls as rounds grow, so this sample cannot separate adaptation to the development shots from the effect of the longer runs; normalising the refusal rate by the number of rounds does not separate them either, and we do not read it as evidence for or against overfitting. The development receipts were written by an earlier version of the producer; re-run with the producer of the held-out sample, all 80 held-out receipt files came back byte for byte, as did 310 of the 320 development files, and on 1 of the 11 development shots whose packing changed,

TABLE III. Willow configurations under the tolerance rule of the original runs and under the $\varepsilon$-checker with $\varepsilon_{\max} = 10^{-6}$, on the same shots. $N$ counts nontrivial shots; the ladder is that of the generation each configuration is counted at (Sec. V); "exact" counts the shots proven with $\varepsilon = 0$. Proven fractions are rounded down.

| configuration | ladder | $N$ | historical rule (%) | $\varepsilon$-checker proven (%) | exact | not proven | largest proven $\varepsilon$ |
|---|---|---|---|---|---|---|---|
| $d = 3$, 13 rounds, X | base | 9,841 | 100.00 | 100.00 | 8,427 | 0 | $4.9 \times 10^{-14}$ |
| $d = 3$, 13 rounds, Z | base | 9,560 | 100.00 | 99.98 | 7,977 | 1 | $4.1 \times 10^{-14}$ |
| $d = 3$, 30 rounds, X | base | 9,994 | 100.00 | 99.98 | 8,359 | 1 | $8.5 \times 10^{-14}$ |
| $d = 3$, 30 rounds, Z | base | 9,991 | 99.98 | 100.00 | 8,119 | 0 | $8.4 \times 10^{-14}$ |
| $d = 5$, 13 rounds, X | base | 10,000 | 99.67 | 99.75 | 3,738 | 25 | $1.9 \times 10^{-13}$ |
| $d = 5$, 13 rounds, Z | base | 9,998 | 99.71 | 99.78 | 3,705 | 21 | $2.6 \times 10^{-13}$ |
| $d = 5$, 30 rounds, X | base | 10,000 | 98.75 | 99.28 | 898 | 72 | $5.9 \times 10^{-13}$ |
| $d = 5$, 30 rounds, Z | base | 10,000 | 98.96 | 99.21 | 801 | 79 | $6.8 \times 10^{-13}$ |
| $d = 7$, 13 rounds, X | base | 10,000 | 98.30 | 99.01 | 933 | 99 | $2.3 \times 10^{-12}$ |
| $d = 7$, 13 rounds, Z | base | 10,000 | 98.49 | 99.03 | 858 | 97 | $1.1 \times 10^{-12}$ |
| $d = 7$, 30 rounds, X | full | 50,000 | 99.99 | 99.99 | 173 | 1 | $1.2 \times 10^{-11}$ |
| $d = 7$, 30 rounds, Z | full | 50,000 | 99.99 | 99.99 | 169 | 1 | $1.4 \times 10^{-11}$ |

the verdict changed from $\varepsilon$-optimal to not proven.

The second protocol (`docs/PREREG_heldout_d7_2026-11-03.md`) addresses the configuration most exposed to fitting: generations 5 and 6 were introduced against the distance-7 shots over 30 rounds, every one of the 50,000 per basis, and no unused distance-7 shot exists at 30 rounds in the archive. It certifies, with the full ladder frozen, the first 2,000 shots per basis at 50 rounds (primary) and at 10 rounds (a bracket on the other side of 30), and, because the development rate came from a producer since optimised, the first 2,000 development shots per basis at 30 rounds with the same producer. It fixes in advance the exact one-sided Clopper–Pearson bound on each not-proven rate, a one-sided conditional binomial test of the held-out rate against the same-producer development rate (which it calls exact; it is exact for two Poisson counts, while for two binomial samples of fixed size the exact conditional test is Fisher's, reported beside it below), how each outcome is to be read, and a stopping rule: after 40 core-hours the run stops at a chunk boundary and every completed chunk is reported as a partial sample.

Its first chunks at 50 rounds cost about ten times the calibrated upper estimate, so with the arms run one after another the rule would have stopped the run inside the first. A deviation committed before any not-proven count was read (`docs/PREREG_heldout_d7_2026-11-03_DEVIATIONS.md`) cycles through the six arms instead and keeps the deadline; Appendix A gives what it was driven by and what it adds to the preregistration, chiefly a per-arm report in three views: the completed chunks (the preregistered one), a worst case that counts every nontrivial shot of an unfinished chunk as not proven, and the longest completed prefix.

The rule stopped the run at 40 core-hours with 43 of the 240 planned chunks complete across the six arms and 3 still being computed (Table IV; `evidence/epsilon_rerun/heldout_d7_stopped_2026-11-04/` keeps both stop records, the watchdog log and the dispatch markers): 2 primary chunks (held-out X 50 rounds seed 9 and held-out Z 50 rounds seed 7, 100 nontrivial shots) and 1 of the same-producer development arm (development Z 30 rounds seed 7, 50 shots). Every completed receipt replays under both verifiers with the forgery battery: 2,150 receipts, 516 forgeries, 86 of them refused by the bound and none accepted.

In the completed chunks, the view the stopping rule names, all 800 held-out 50-round corrections (both bases) were proven within $\varepsilon_{\max}$ of optimal, 0 of them with $\varepsilon = 0$, with a largest proven slack of $3.2 \times 10^{-12}$; at 10 rounds all 700 were (93 with $\varepsilon = 0$), and on the development shots with the same producer at 30 rounds all 650 (3 with $\varepsilon = 0$). The preregistered one-sided conditional binomial test of the held-out 50-round not-proven rate against the same-producer development rate, on the completed chunks of both, does not reject it (one-sided $p = 1.00$; the 10-round bracket gives $p = 1.00$; Fisher's exact test, the exact conditional test for two binomial samples, gives $p = 1.00$ and $p = 1.00$); the preregistration prescribes no further reading for this case.

That is the result: every completed primary shot was proven, 100 primary shots were unfinished at the stop, and no population rate is claimed. The preregistered computation, the exact one-sided 95% Clopper–Pearson bound on each not-proven rate, gives 0.374% at 50 rounds, 0.428% at 10 and 0.460% at 30 (Table IV).

Its coverage guarantee assumes that which shots are counted does not depend on their outcomes, and a stop that keeps the chunks finished inside a budget violates that assumption: a chunk finishes late when its shots need the ladder's expensive tiers, which is where refusals come from. So we do not make the statement the preregistration permits when that bound is below 1%, that the full ladder proved at least (100 − bound)% of held-out 50-round corrections (`docs/PREREG_heldout_d7_2026-11-03_DEVIATIONS.md`, note 2), and the same selec-

tion qualifies the two tests above. Two descriptive views show how much the selection could matter, and neither is a bound with coverage: counting the 100 unfinished primary shots as not proven, 800 of 900 dispatched primary shots were proven, and the longest run of completed chunks from the first holds 700 50-round shots with 0 not proven. A coverage claim would need a cohort frozen independently of completion, a defined per-shot computational budget, and an outcome recorded for every shot, timeouts included.

Even free of that selection, 800 shots with no refusal could not confirm the 99.998% of the abstract, a development-sample rate that a held-out sample could only confirm with about 150,000 shots and no refusal; the recorded development rate of 2026-09-17 (2 not proven of 100,000, bound 0.00630%) is an earlier producer's and is reported beside, never pooled. No per-round normalisation is used for inference; every rate above is per shot, with its number of rounds named.

### C. Is the decoder, or the producer, responsible for refusals?

A refusal can mean the decoder's correction was not optimal or that the producer failed to find a tight packing. To separate the two we computed, for every nontrivial shot of three simulated configurations, a minimum-weight perfect matching on the metric closure over $T$ with per-detector boundary copies, using networkx [12], a different formulation and algorithm sharing no code with the producer. On all 870 shots (370 at distance 3 over 9 rounds, 300 at distance 5 and 200 at distance 7, both over 25 rounds) the oracle's weight equalled the decoder's to floating-point resolution, far inside $\varepsilon_{\max}$, so every refusal on those shots was producer slack; Sec. VII repeats the separation with an exact-arithmetic oracle.

### D. Other hardware

To check that nothing is specific to one vendor's data format or graph construction, we certified a repetition-code memory run on IBM's ibm_fez processor, preregistered before submission (SHA-256 prefix `0a3607c9`), from the job's raw registers, which are released: with the full ladder, all 2,296 of 2,296 nontrivial shots were proven within $\varepsilon_{\max}$ of optimal, 2,257 of them with $\varepsilon = 0$, both verifiers returned the same verdict, slack and bound on every receipt, and none of 21,169 forgeries was accepted, 4,748 of them refused by the bound. The error model is ours, a declared uniform model, so these shots are reported separately from every total above.

### E. Certifying against a model measured from the run itself

Every run above certifies against weights supplied from outside the shots being decoded. A certificate proves minimum weight with respect to the weights it is given and is indifferent to where they came from, so the question here is narrower than it may look: whether the accept path runs unchanged on a model estimated from the device's own syndromes, the form a deployed system would want. The estimator is not ours: the two-point estimator that recovers pairwise error probabilities from detector correlations alone is due to Spitz *et al.* [13], the analysis Google applied to repetition codes [14], and Krishnamoorthy *et al.* have decoded IBM hardware shots under "a *learned* model on the same graph whose priors are re-fitted from the recorded detector statistics by the standard two-point correlation estimator" ([15], Sec. 8.6: a distance-3 surface code over three rounds on `ibm_kingston`). There the re-fitted model made the logical flip rate slightly worse (0.253, against 0.246 for the best decoder under the calibration model), and the certificate rate reported, "the AIS decoder certifies 96% of its decisions", belongs to the decoding under the calibration model; those certificates assert maximum-likelihood optimality under the assumed model and are statistical, a paired bootstrap on annealed-importance-sampling weights (their Sec. 7.4).

We estimated a decoding graph from the first half of each run's detectors and certified the second half against it, on 6 preregistered hardware records from 2 IBM processors, `ibm_kingston` (2 records) and `ibm_fez` (4 records), at distances 3 and 5 and at 13 and 40 rounds, 70,000 held-out shots in total, on graphs of 168 to 573 edges. Of 40,979 nontrivial held-out shots the exact checker proved 40,963 exactly optimal (the 16 refusals all on the two 40-round records) and the $\varepsilon$-checker proved 40,979 within $10^{-6}$. Every receipt is kept with its estimated graph (`evidence/certified_measured_receipts/`), both verifiers and the JavaScript checker return the same verdict on every one, and none of 462,794 forgeries was accepted, 122,767 of them refused by the bound. The accept path and the campaign's own scorer return identical logical-error counts on every record.

We do not report how much better the estimated graph decodes than the models it replaces: on these short repetition-code records the baseline models' logical error rates are poor enough that the comparison would say more about the baselines than about the estimator, and estimated models outperforming device-informed ones is documented elsewhere, on Willow and on IBM hardware [16]. A neural decoder conditioned on calibration data, not on a model estimated from syndromes, reports, on IBM repetition-code data from Fez, Kingston and Pittsburgh, logical error rates up to 11.11× lower than calibration-informed matching at distance 11 on validation data pooled across the three processors, and 7.41× (Z basis) and 2.09× (X) lower on unseen Kingston

TABLE IV. The held-out distance-7 sample as its preregistered stopping rule left it, per arm and pooled over the two bases: chunks completed of 40 planned, nontrivial shots $N$, proven within $\varepsilon_{\max}$, not proven, the preregistered computation of the exact one-sided 95% Clopper–Pearson bound on the not-proven rate (prereg. CP, %), which carries no coverage guarantee for a sample selected by completion (Sec. VI), the longest run of completed chunks from the first (prefix), and the nontrivial shots of the chunks still being computed at the stop (unfinished), which the worst case counts as not proven. Rounds are named with every rate.

| arm | rounds | chunks | $N$ | proven | not proven | prereg. CP (%) | prefix | unfinished |
|---|---|---|---|---|---|---|---|---|
| held-out X | 50 | 10/40 | 500 | 500 | 0 | 0.598 | 8 | 50 |
| held-out X | 10 | 7/40 | 350 | 350 | 0 | 0.853 | 7 | 0 |
| development X | 30 | 7/40 | 350 | 350 | 0 | 0.853 | 7 | 0 |
| held-out Z | 50 | 6/40 | 300 | 300 | 0 | 0.994 | 6 | 50 |
| held-out Z | 10 | 7/40 | 350 | 350 | 0 | 0.853 | 7 | 0 |
| development Z | 30 | 6/40 | 300 | 300 | 0 | 0.994 | 6 | 50 |
| held-out, pooled | 50 | 16/80 | 800 | 800 | 0 | 0.374 | – | 100 |
| held-out bracket, pooled | 10 | 14/80 | 700 | 700 | 0 | 0.428 | – | 0 |
| development, pooled | 30 | 13/80 | 650 | 650 | 0 | 0.460 | – | 50 |

chains [17]. No priority is claimed for the estimator or for any decoding margin it brings. Certifying against a measured model does not make the measured model true; it makes the decoder's answer checkable under that model, and nothing more.

## VII. WHAT THE CERTIFICATE DETECTS

### A. Seeded decoder faults

What a certificate can detect is fixed in advance by the lemma: a packing proves at most the optimum, so no packing certifies a correction heavier than the optimum by more than $\varepsilon_{\max}$, and a sound checker refuses every such correction. The experiments below therefore do not measure detection power in the usual sense. They test that the implemented checker behaves as the lemma says on realistic faults, and they measure what the lemma cannot fix: the false-alarm rate, the refusals of corrections that are in fact within $\varepsilon_{\max}$ of optimal. The lighter correction that proves a refused one suboptimal comes from an exact oracle, not from the certificate; a certificate alone says "not proven", and only a lighter valid correction says "suboptimal".

We mutated the decoder's output on 300 shots at each of distances 3, 5 and 7, which models wrong answers but not crashes, hangs, or memory errors, in eight ways (`experiments/m0_prior_art/decoder_mutation_gate.py`; counts summed over the distances): the last edge replaced by an edge the correction lacks (770), a matched pair broken by sending one of its detectors to the boundary (656), the cheapest edge dropped (770), the previous shot's correction returned (766), the second-to-last edge dropped (587), an even cycle edge-disjoint from the correction added (770), one edge replaced by a heavier two-edge detour through a common neighbour (708), and two boundary edges replaced by the heavier direct edge between their detectors (3). Each mutant is the edge set it denotes, and one whose edge set is the decoder's own correction is discarded (Appendix A). Every correction was judged against the exact minimum-weight correction, computed by an exact-arithmetic oracle whose answer is re-verified before use. Each receives one outcome under the $\varepsilon$-checker: *invalid*, when it does not explain the syndrome; *certified*; *proven suboptimal*, when it is refused and a lighter valid correction is exhibited; or *not proven*, when it is refused without such a witness. Every one of the 5,030 mutants is defective: 3,549 were invalid, which syndrome checking also detects, and the other 1,481 were proven suboptimal; none was certified and none was left not proven. Those 1,481 are the informative subset: added cycles, detours and boundary-pair swaps, which leave the syndrome exactly satisfied while making the correction strictly heavier, invisible to syndrome checking by construction. On the 770 unmodified corrections of the same shots, the base ladder refused 1 (0.00328–0.722%) and the full ladder 0 (at most 0.389%), and every such refusal was producer slack on a correction within $\varepsilon_{\max}$ of optimal; these are the false-alarm measurement, since the construction meant as a control on equal-weight alternatives, every fired detector sent to the boundary when that ties the correction's weight, gave no correction other than the decoder's own on these shots (Appendix A). The released corpus holds the 5,030 seeded faults, so other detectors can be scored on the same faults.

This experiment is open to the objection that the mutations were chosen knowing what the certificate checks. The next one is not, although it needs a reference of its own.

### B. A stale calibration

Here the decoder runs unmodified and its output is never touched. The fault is upstream: the decoder decodes on a graph built from a drifted error model, as it

TABLE V. Stale-calibration fault at drift factor 3.0, judged against the exact optimum under the reference model. Every correction is syndrome-consistent. False alarms are refusals of corrections within $\varepsilon_{\max}$ of optimal. The last three columns count shots on which only the stale decoder, only the reference decoder, or both made a logical error.

| | | proven | false | logical errors | | |
|---|---|---|---|---|---|---|
| | $N$ | subopt. | alarms | stale only | ref. only | both |
| $d = 3$ | 452 | 24 | 0 | 1 | 1 | 6 |
| $d = 5$ | 797 | 76 | 0 | 4 | 0 | 12 |
| $d = 7$ | 800 | 171 | 0 | 3 | 1 | 4 |

would if its calibration snapshot were stale, and its corrections are checked against a reference model supplied independently, here the model that generated the data. A drift factor $f$ multiplies each error mechanism's probability by an independent factor $e^U$, with $U$ uniform on $[-\ln f, \ln f]$, clipped to $[10^{-12}, 0.49]$, and the decoder's matching graph is rebuilt from the drifted model. Every correction is syndrome-consistent by construction, because the decoder solves a correct problem with wrong weights.

What the certificate adds deserves a precise statement. With the reference model in hand, comparing the two models detects that they differ, without any certificate. What it does not say is which decisions the difference changed, or by how much. The certificate answers that per shot, without running a second decoder: it flags exactly the corrections the reference does not prove near-optimal. Turning a flag into "proven suboptimal" still needs a lighter correction, which here comes from the exact oracle under the reference, and the certificate cannot say which of the two models describes the device.

Across drift factors 1.5, 2.0 and 3.0, the stale decoder produced 110, 166 and 271 suboptimal corrections. Syndrome checking detected none of them and every one was proven suboptimal. Of the 6,147 corrections at those drifts (the three distances, 2,049 at each drift), 5,600 were within $\varepsilon_{\max}$ of the reference optimum, and the certificate raised 2 false alarms on them. The drift factors share their base shots, and the 2 fell on a single base shot, the one that also raised the false alarm of the drift-1.0 control: they are one shot seen at more than one drift factor, not independent trials, so we give no sampling interval for the pooled count. Severity scales with drift: the mean excess weight of the suboptimal corrections, over the optimum under the reference model, rises from 0.13–0.25 at drift 1.5 to 0.39–0.64 at drift 3.0, each range spanning the three distances. Logical error counts barely register the fault: at distance 3 and drift 3.0 the reference and stale decoders commit 7 and 7 logical errors while 24 corrections are proven suboptimal, and over all three distances at drift 3.0 the discordant shots split 8 against 2 (exact two-sided binomial $p = 0.11$; Table V). As a control, drift factor 1.0 produced 0 suboptimal corrections at every distance (false alarms: 1).

## VIII. PER-SHOT OPTIMALITY GAPS

Decoders are usually compared by logical error rate, an average that cannot distinguish a decoder that is consistently slightly suboptimal from one that is usually exact with a heavy tail. A certified correction on the same shot makes each decoder's excess over the minimum measurable, and we measured it in an experiment fixed before it ran (`docs/PREREG_gap_distribution_2026-11-03.md`): 2,000 shots per distance $d \in \{3, 5, 7\}$, rounds equal to distance, $p = 0.005$, from seeds 4108103, 4108105 and 4108107 that no earlier run used.

The reference is PyMatching's correction $J$, proven by the base ladder and the $\varepsilon$-checker within $\varepsilon(J) \le 10^{-6}$ of optimal on 1,159 / 1,978 / 1,996 of 1,159 / 1,979 / 2,000 nontrivial shots. The optimum then lies in $[w(J) - \varepsilon(J), w(J)]$, so for another decoder's correction $K$ the excess $w(K) - w(J^\star)$ lies in $[\max(0, g),\ g + \varepsilon(J)]$ with $g = w(K) - w(J)$, computed in exact rationals of the binary64 weights the checker reads. $K$ is *proven within $\varepsilon_{\max}$ of optimal* when the upper end is at most $10^{-6}$ and a *proven miss* when the lower end exceeds it; no shot fell between. A decoder's selection is projected onto graph edges modulo 2 before it is weighed, since a correction is an edge set. Three decoders were run. BP-OSD [18, 19] (ldpc 2.4.1 [20] `BpOsdDecoder`: min-sum, 30 iterations, parallel schedule, OSD-CS of order 5) in two settings: one column per matching-graph edge, with the XOR-combined probability of the graphlike components on it, which reproduces PyMatching's merged weights to $5.1 \times 10^{-16}$ relative and so gives BP-OSD the matching graph's objective, with min-sum scaling 0.625 (the preregistered primary arm); and one column per graphlike component of each decomposed error, components treated as independent, with the library's default unscaled min-sum (the setting of this section's earlier version, now a sensitivity arm, which the preregistration calls an input-model one). The two BP-OSD arms therefore differ in two factors at once, the input model and the min-sum scaling, and comparing them compares two settings; a post hoc $2 \times 2$ below separates the factors. And belief-matching [21], which matches under weights informed by belief propagation. Neither BP-OSD setting is a maximum-likelihood decoder, and a gap is weight, not homology class.

Table VI gives the result. BP-OSD on the merged edge model was proven within $10^{-6}$ of the minimum on 99.48 / 99.24 / 95.69% of shots at $d = 3$, 5 and 7, belief-matching on 95.25 / 85.64 / 62.27%, and BP-OSD on independent components on 91.28 / 85.38 / 79.45%; their 99th-percentile excesses are 0.0 / 0.0 / 1.0, 2.8 / 3.7 / 5.9 and 2.6 / 2.0 / 2.6, and their largest 4.0, 16.9 and 12.4 weight units. On these data the primary setting, BP-OSD on the merged edge model with scaling 0.625, lands on the minimum-weight correction more often than belief-matching does and more often than the component setting (unscaled, on independent components) at every distance, and the component setting's 99th-percentile excess

TABLE VI. Excess weight over the minimum on the preregistered shots, against PyMatching's correction proven within $\varepsilon(J) \leq 10^{-6}$ of optimal ($N$ shots). "Proven within" counts corrections whose excess is proven at most $\varepsilon_{\max} = 10^{-6}$; a proven miss is proven to exceed it. The excess columns are the upper ends of the exact intervals $[\max(0, g), g + \varepsilon(J)]$.

| | | | proven within $\varepsilon_{\max}$ of optimal | | proven miss | excess (upper end) | | |
|---|---|---|---|---|---|---|---|---|
| $d$ | decoder | $N$ | % | 95% interval | beyond $\varepsilon_{\max}$ | median | p99 | max |
| 3 | BP-OSD, merged, $s = 0.625$ | 1,159 | 99.48 | 98.87–99.81 | 6 | 0.00 | 0.00 | 0.04 |
| | BP-OSD, components, $s = 1$ | 1,159 | 91.28 | 89.51–92.85 | 101 | 0.00 | 2.63 | 11.83 |
| | belief-matching | 1,159 | 95.25 | 93.86–96.41 | 55 | 0.00 | 2.80 | 6.87 |
| 5 | BP-OSD, merged, $s = 0.625$ | 1,978 | 99.24 | 98.75–99.58 | 15 | 0.00 | 0.00 | 3.15 |
| | BP-OSD, components, $s = 1$ | 1,978 | 85.38 | 83.75–86.92 | 289 | 0.00 | 1.96 | 6.55 |
| | belief-matching | 1,978 | 85.64 | 84.01–87.16 | 284 | 0.00 | 3.71 | 16.87 |
| 7 | BP-OSD, merged, $s = 0.625$ | 1,996 | 95.69 | 94.70–96.54 | 86 | 0.00 | 1.03 | 4.02 |
| | BP-OSD, components, $s = 1$ | 1,996 | 79.45 | 77.61–81.22 | 410 | 0.00 | 2.62 | 12.38 |
| | belief-matching | 1,996 | 62.27 | 60.10–64.41 | 753 | 0.00 | 5.89 | 11.73 |

is larger at every distance. None of the three decoders returned a syndrome-inconsistent correction on any shot. Belief-matching (run with `max_bp_iters = 20`) does not aim at the minimum-weight correction under these weights: it matches under weights re-estimated by belief propagation, so its excess measures how far its objective is from the reference's, not how often it fails. Scored instead by whether the observable parity of each projected edge set matches the shot's true flip, the logical failures on the nontrivial shots were 28 / 27 / 15 for PyMatching, 28 / 26 / 16 for BP-OSD merged, 32 / 26 / 18 for BP-OSD on components and 29 / 12 / 7 for belief-matching at $d = 3$, 5 and 7 (`gap_distribution_logical.py`). On the exact-checker subset, which is smaller (1,154 / 1,918 / 1,282 shots) and keeps easier shots, the ordering is the same, with exact hits on 99.48 / 98.95 / 96.41%, 95.23 / 85.71 / 65.75% and 91.33 / 85.24 / 82.13%. Fig. 3 shows the tails. The record keeps, per shot, the syndrome, each decoder's selection, its projected edge set and exact cost, and the reference verdict, which the checker as tightened on 2026-11-03 reproduces on all 5,138 shots.

*Which factor, post hoc.* The comparison of the two BP-OSD arms above is between settings. To attribute it we ran the two missing cells of the $2 \times 2$ after the fact, on exactly the preregistered shots and scored the same way against the same reference (`gap_distribution_2x2_posthoc.py`, which refuses to report unless it first reproduces both preregistered arms shot for shot): BP-OSD on merged edges with unscaled min-sum was proven within $10^{-6}$ of the minimum on 99.05 / 98.73 / 97.34% of shots at $d = 3$, 5 and 7, and on independent components with scaling 0.625 on 98.27 / 91.55 / 80.91%. At either scaling the merged model is proven within $\varepsilon_{\max}$ more often than independent components at every distance (exact McNemar test on the shots where the two disagree, $p \leq 0.0094$). Scaling 0.625 helps the component model at every distance ($p \leq 0.012$); on merged edges it makes no significant difference at $d = 3$ and 5, and at $d = 7$ the unscaled arm is proven within more often, with a heavier tail ($p = 0.33/0.14/0.0018$ at $d = 3$, 5 and 7; 99th-percentile excess 13.5 against 1.0 at $d = 7$). Of the difference between the two preregistered arms, the input model accounts for more than half at $d = 5$ and 7 whichever factor is changed first, while at $d = 3$ changing either factor alone recovers more than half of it. The mechanism this section's earlier version gave the representation belongs to the scaling: on independent components unscaled BP-OSD selected several parallel columns of one edge on 188 / 417 / 126 shots, typically a converged belief-propagation hard decision that flips every copy of a group of identical columns, and at scaling 0.625 on 1 / 0 / 0; merged edges have one column per edge, so it cannot occur there. What remains of the component model's deficit at scaling 0.625 is the input model's. These comparisons are post hoc and not corrected for multiplicity, and their record is kept apart from the preregistered one (`evidence/gap_distribution_v4/posthoc_2x2_2026-11-06/`).

*What this section's earlier version reported.* Its scorer charged an edge's weight once per selected component instead of once per edge of the projected correction. Rerunning both earlier experiments exactly as they were run, with every selection kept and each stored artifact reproduced bit for bit, 158 of the 1,000 shots of its table and 149 of the 863 of its figure were overcounted, by 7.62–146.31 weight units each, one edge being selected up to 15 times; 95 of them were in fact exactly optimal. Rescored, its largest gap of 148 weight units becomes 20.7, its 99th percentiles of 38 / 61 / 52 become 2.6 / 2.6 / 1.7, and its finding that belief-matching lands far closer to the minimum than BP-OSD does not survive. It also called a gap exact when its magnitude was below $10^{-9}$, which counted 17 nonzero gaps as zero, 11 of them negative: BP-OSD lighter than a correction the tolerance rule had accepted.

## IX. COST

We timed the base ladder on the configuration of the simulation at scale, distance 5 over 25 rounds at $p = 0.005$, on 2,000 nontrivial shots from that run's

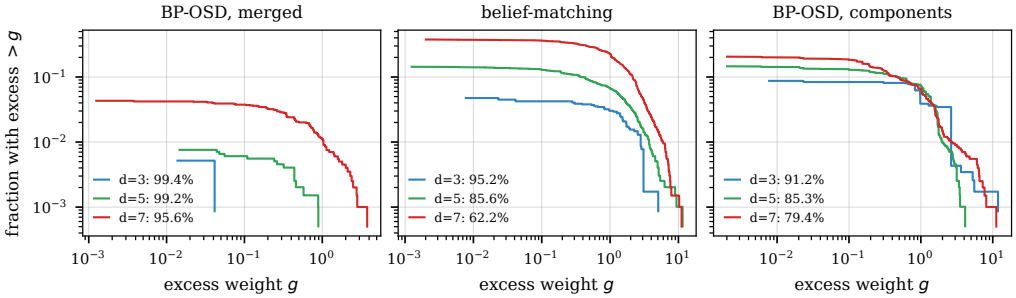



FIG. 3. Fraction of preregistered shots whose excess weight over the minimum (upper end of the exact interval) exceeds $g$, per decoder and distance; BP-OSD merged runs at min-sum scaling 0.625 and BP-OSD on components unscaled, so the two differ in setting, not only in input model. The remaining mass is proven within $10^{-6}$ of optimal; the legend gives its size.

own seed plan; every verdict matched the stored receipt for the same shot. Certification took a median of 30 ms per shot, a mean of 53 ms and a 99th percentile of 275 ms; the 4 refused shots took a median of about 400 ms each. Against PyMatching on the same graph, a mean of 16.4 $\mu$s per shot batched and 132 $\mu$s per call, the mean cost of certification is about 3,200 and 400 times the mean cost of decoding. The per-call figure is PyMatching's `decode_to_edges_array` on the shot's detector vector, called right after the previous shot's certification. A second run on the same shots (`experiments/m0_prior_art/decode_call_context.py`) timed that call at a mean of 110 $\mu$s, the same call repeated at once on the same shot at 31.9 $\mu$s, and the compiled graph's decode called with the fired detectors' indices, the call timed below for the decoder's own dual, at 19.3 $\mu$s; that run's mean certification, 52.6 ms, is about 480, 1,600 and 2,700 times these. We did not measure why a decode made after a certification is slower; the per-call ratio of about 400 is against that slower call. Checking a certificate with the $\varepsilon$-checker took a mean of 7.4 ms (median 7 ms, 99th percentile 17 ms), about 450 times the mean batched decode. These are single-thread Python times on a shared workstation whose processors were 70–74% busy when sampled (43–74% in the second run); each ratio was measured within one run.

*What the decoder's own dual gives.* Most of that cost is the producer's, not the problem's. Sparse blossom maintains, and then discards, a dual of exactly the form the checker reads: its frozen region radii are nested $T$-odd sets whose values sum to the matching weight. A patch that adds 125 lines to 3 files of PyMatching 2.4.0 and removes none (`third_party/pymatching_return_duals.patch`, applied to the PyPI source distribution with `patch -p1` or `git apply`) exports it. We rebuilt it on the machine of this paper and ran it on 2,000 shots of the simulation's seed plan and the first 2,000 and 2,000 shots of two Willow development configurations (distance 5 and 7 over 30 rounds, X), in two arms (`experiments/m0_prior_art/pymatching_native_dual.py` and `pymatching_native_dual_arms.py`). In the first, PyMatching decodes the published weights, which it discretises internally to the integers $2\,\mathrm{round}(wc)$, $c = (2^{24}-1)/\max_e w_e$; checked exactly against those integers, its exported dual proves every one of the 6,000 corrections exactly optimal, and the scale the decoder reports equals $2c$ on every shot. In the second, we rounded the weights ourselves onto a dyadic grid ($2^{-19}$, $2^{-18}$, $2^{-18}$) and scaled them to integers, so that PyMatching's integer problem is the grid problem; there too its dual proves every correction exactly optimal. On every shot the two arms return the same correction, and it is the correction this paper certifies for that shot; as a control, the certified correction with one edge dropped is told apart from both on every shot. Neither dual is a proof on the published doubles. Divided by its scale and judged there, the first arm's dual has proven slack with medians $4.0\times10^{-6}$, $1.0\times10^{-5}$ and $2.0\times10^{-5}$ and a maximum of $3.6\times10^{-5}$, so under $\varepsilon_{\max}=10^{-6}$ it proves 3, 0 and 0 of the shots; the second arm's dual, judged there, has medians $2.6\times10^{-5}$, $5.0\times10^{-5}$ and $1.1\times10^{-4}$, a maximum of $1.9\times10^{-4}$, and proves 0. Per call, on the same calls, we timed the witness's production, its check and the two together. Decoding with the export and turning its regions into cut sets in Python, which produces the witness, took a median of 115 $\mu$s (99th percentile 230 $\mu$s) against 17 $\mu$s (40 $\mu$s) for a plain decode of the same shot on the simulation, and 374 $\mu$s (627 $\mu$s) against 63 $\mu$s (128 $\mu$s) at distance 7: 5.9–6.9 times a plain decode across the three configurations, and 6.6–8.2 in a run of the same code on the same shots on 2026-11-03, which did not time the check and whose record agrees with this one in every field that is not a time. Checking the witness exactly against the decoder's integers with our Python checker, in rational arithmetic, took a median of 5.1 ms (99th percentile 13.6 ms) on the simulation and 14.6 ms (30.4 ms) at distance 7, so the complete call, decoding with the export, conversion and exact check, took 5.2 ms (13.7 ms) and 14.9 ms (30.8 ms), 236–311 times a plain decode. The first record's export plus conversion (`pymatching_native_dual.py`) came to 11.0–12.8 times a plain decode; its conversion also rescaled every value into binary64 weight units, which the claim on the decoder's integers does not need. These are single-thread medians and 99th percentiles over single calls on a shared workstation. Their plain decode is the compiled graph's

call, made right after the same shot's decode with the export and here with the patched build: the context of the fastest of the three calls timed above, not that of the cost paragraph's per-call mean, a decode made after a certification. A decoder-native dual therefore yields a witness of exactness for the integers the decoder actually decodes with at 5.9–6.9 times a plain decode per call, and on this route the exact check, as our Python checker does it, is most of the cost of the certified call (on the decoder's integers the same check needs no rationals; no faster checker was measured); the linear-programming ladder is what buys $10^{-6}$ on the published doubles, and its cost ratios above are a property of that producer, not of certification. With current producers at $\varepsilon_{\max} = 10^{-6}$, certification suits offline audit, acceptance testing of decoder releases, and spot-checking a sampled fraction of a live stream.

## X. RELATED WORK AND SCOPE

*Certifying and checking.* Certifying algorithms [2] and program checkers [3] are classical, as is the linear-programming structure of matching and $T$-joins [1, 22]. In exact integer programming, the gap between a floating-point solver's claim and a proof is well studied, with safe dual bounds [23, 24] and verifiable certificates [25, 26]; Sec. IV addresses the same issue for decoding receipts, and its lemma is a safe bound of that family in which scaling a non-negative packing takes the place of correcting a dual's residual.

*Decoding.* Wu *et al.* [27] formulate most-likely-error decoding as a minimum-weight parity factor problem and decode with a primal–dual algorithm that emits a proximity bound with every solution, which certifies the solution optimal when what they call the Certifiability Condition holds, that the linear program's minimum equals the integer program's. Exact matching codes have long exposed their dual: Kolmogorov's Blossom V returns its dual solution after solving and provides a check of complementary slackness against it [28]. That is the dual of perfect matching on the graph it is given, with node values free in sign, which bounds a minimum-weight $T$-join only when that graph is the metric closure over $T$ with the boundary represented, as in Sec. VI C; Sec. IX shows that PyMatching's sparse blossom holds a dual of the $T$-join form itself. What differs here is the setting: a certificate for a third-party decoder's correction, decided by separately written checkers in exact arithmetic over the published weights, with the cost of producing it measured. Linear-programming decoding of classical and quantum codes has a long history [29, 30].

*Concurrent work.* Krishnamoorthy *et al.* [15] decide degenerate-coset maximum likelihood on surface and bivariate-bicycle codes. Their certificates are of three kinds: a paired bootstrap test, which is statistical; an exact decision when each class's estimate carries a constant-factor guarantee (their Lemma 1), as from the WISH estimator, a route they test only on the distance-5 rotated surface code, with an exact oracle, and whose use with a quantum annealer as the oracle they call "a concrete, if currently speculative, possibility"; and deterministic certificates from mini-bucket bounds. On the $[[72, 12, 6]]$ code their exact decoding costs 5 to 26 s per syndrome, and mini-bucket elimination at $i$-bounds 16 and 20 reproduces the exact decision on 40 of 40 syndromes and deterministically certifies 37 and 38 of them at a fraction of that cost; on the $[[144, 12, 12]]$ code their mini-bucket run produced no deterministic certificates, at about 272 s per decode. Their objective is coset maximum likelihood, not the minimum-weight correction certified here, so the two kinds of certificate answer different questions. Shen and Zhong [31] test decoder libraries for conformance and release an oracle-free suboptimality certificate, two feasible corrections of different weight, re-derived by a standalone verifier; that is the witness this paper calls "proven suboptimal" (Sec. VII), and the lower bound that certifies optimality is what the receipts here add to it. Ehatamm *et al.* [32] formalise stabilizer-code theory in Lean 4 and obtain Lean-checked distance proofs of individual codes, among them the $[[90, 8, 10]]$ and $[[70, 6, 9]]$ bivariate-bicycle codes; their object is a code's distance, not a decoder's output. Our literature searches, whose queries are logged with the released artifacts, did not find per-shot $\varepsilon$-optimality receipts for a matching decoder on hardware data at this scale, kept and replayed; not finding them is not proof of absence.

*What is not claimed.* The rates are properties of our producer, a decoder and an error model: they measure how often the producer proves the decoder's answer near-optimal for the given weights, not logical accuracy, hardware fidelity, or real-time readiness; soundness is a property of the checker alone. Certification with our producer is not a real-time decoder (Sec. IX). It is not complete: 0.19% of simulated corrections and 397 hardware corrections were not proven, and those receive receipts stating their proven slack. Exact optimality is counted separately and not claimed at scale. A certificate proves optimality for the weights it was given and says nothing about whether those weights describe the device. Nor can it separate tied optima: a decoder change that selects a different equal-weight correction, possibly in a different logical class, selects another optimum, which a packing proves as well as the first, so it passes unchanged.

## XI. REPRODUCIBILITY AND DATA

Code, artifacts, conformance vectors, the seeded-fault corpus and the certificate schema are released under the Apache License 2.0 at `https://github.com/coherence-energy-labs/oneq-certificates` (release v2026.11.07.2). The receipts of the simulated, Willow and held-out runs of Sec. VI are release assets: one uncompressed tar file per configuration, 24 in all, each holding that configuration's receipts as gzip-

compressed chunk files. Each tar's SHA-256 and size are pinned in its configuration's `archive.json`, and each chunk's SHA-256, taken over its decompressed bytes, in the chunk index beside the configuration's summary. With the archives unpacked, one command, `python tools/replay_epsilon_receipts.py`, re-decides every receipt of the 18 complete configurations with both Python verifiers and checks every chunk against its index and every summary against its receipts; `--battery 1` and `--forgeries 1` add the attacks of Sec. VI. The six arms of the held-out distance-7 sample, stopped by their budget and without summaries, need a second command, with `--partial` and the six arms named (`REPRODUCE.md`). The receipts of Secs. VI D and VI E are not release assets but files in the repository (`evidence/ibm_live/`, `evidence/certified_measured_receipts/`): both verifiers decided every one when it was produced, the generator of this paper's numbers checks their hashes, and the shipped tests re-decide a sample of the latter. The JavaScript checker replayed a sample, not every file: the first 4 chunks of each of 8 released configurations (at most 1,000 receipts each) and all 40,979 receipts of Sec. VI E, the 48,969 of the abstract. Every measured number in this paper is typeset from a macro generated from a JSON artifact, a gate in the repository traces every number in the source to the evidence tree, and each artifact records the command and the bytes of the code that produced it; a number is typeset only from bytes that a commit holds. The Willow data are read in place from the published archive [11].

Inputs are platform-dependent at the last bit. With identical package versions, PyMatching's merged weights differed in the last place on several edges of the distance-3 graph between a Linux machine and the Windows machine of this paper, and Stim's seeded sampler does not reproduce samples across platforms. Every receipt therefore ships with its graph, an instance file whose hash the receipt's summary carries, and every per-shot record of Sec. VIII with its syndrome; a replay never rebuilds either.

*Which repository.* The commits this paper names are commits of the project's development repository, which is private. The public repository holds the releases with a squashed history, so none of those commits can be opened there, and neither can the commit dates that put each preregistration before its run (Sec. VI). What a reader can check is what was released. Every commit this paper names ships as a release tarball `oneq-source-<commit>.tar`, pinned by SHA-256 in `evidence/revision_snapshots.json`: for the four commits of Table I, the producing scripts and every module they import; for each other commit, the files its citation is about and, where it is cited for a change, that commit's diff of them. For the other runs, the files each was judged by ship as the bytes that ran, with their SHA-256 in the run's record, or are named by the generator, when it runs outside the development repository, where a later revision changed them. The full history is available to referees on request. A commit's date is set by whoever makes the commit, so these timestamps can be trusted only as far as the author is. Future preregistrations will be anchored outside the project (`docs/PREREG_ANCHORING.md`): an anchor file naming the preregistration's hash and a future round of a public randomness beacon, with an OpenTimestamps proof attested in a Bitcoin block before that round is published, and the run's seeds drawn from that round.

*Use of AI tools.* AI tools assisted with software development and manuscript preparation, and wrote the third checker of Sec. III from a logged brief. Reviews of this manuscript by AI agents that had not written it are archived with it (`docs/reviews/`), as are the four external audits of its releases of November 2026: the third calls itself AI-assisted, the fourth states that it was done by an AI model with AI sub-agents, and the first two do not say who or what produced them. None of these is third-party peer review. The author is responsible for all content.

## Appendix A: Verification log

This appendix records how the evidence above was produced and every defect found in it, so that the body can state each claim once.

*A dual that looks right and is not.* Node potentials checked edge by edge, $y_u + y_v \le w_e$, bound a $T$-join only in the special case of Sec. II, non-negative values on the vertices of $T$, which are a packing of singletons. Once a potential may be negative or sit on a vertex outside $T$, the edge constraints no longer bound the sum: a negative value on an unfired vertex relaxes the constraint of every edge at it and is paid for once, so with two or more fired neighbours the sum can exceed the optimum, and a positive value outside $T$ adds to the sum what no $T$-join need pay. In development, node potentials run against PyMatching at distance 7 gave such a "bound", above the weight of the correction it was shown, so they were not of the valid form. The checker refuses any bound above the primal as impossible, and the exact checker and the $\varepsilon$-checker refuse nonzero node potentials outright.

*A repeated set.* Writing the first JavaScript checker exposed an ambiguity: given a packing that listed the same vertex set twice with different values, the Python checker deserialised it into a dictionary, kept the last value and accepted, while the JavaScript checker summed the list, found an edge constraint violated, and refused. Both now refuse a repeated set, and the case is a permanent conformance vector.

*What of the first JavaScript checker is independent.* From the history of the development repository, which is not public (Sec. XI): it was written from the specification on 2026-07-31 (commit `7a5a3cff`); the exact lattice rule was ported from the Python checker on 2026-08-04 (`888b7454`); the dual repair that replaced it, and its later spread-absorb and four-order variants, were written

into both languages in the same commits on 2026-08-05 (`6b3a146d`, `e0767dc7`, `900d6471`), deliberately identical; and boundary-first edge keys failed to canonicalise in both until `56dce450` (2026-08-18). Its header, which said nothing in it was ported, now says this.

*A defect in the exact checkers, found and fixed.* Until 2026-10-18 the exact checker (`matching_cert.check`), in every version including the one that applied the tolerance rule in the original runs, never tested that every set of a packing is $T$-odd; a stricter variant tested oddness against the fired detectors alone, without the boundary that $T$ includes when their number is odd. Forged certificates that a linear-programming search over small graphs constructed exploit both: packings over sets that are not $T$-join cut constraints, certifying corrections of weight 6, 7 and 5 against optima of 4, 6 and 4. The exact checker accepted all three and the stricter variant the first two; both were fixed in commit `617da3af`, and the three are conformance vectors. The JavaScript checker always refused them, so the two implementations had disagreed on these inputs, and no vector contained one. The $\varepsilon$-checker has tested $T$-oddness, with $b$ counted, since its first version, so no headline rate depends on the defect. Every count an exact checker decided was recomputed with the fixed checkers and is unchanged: the exact-optimality columns of Table I and the certified counts of Sec. VI E.

*The input domain, made one policy.* The differential fuzzer of Sec. III found the Python readers coercing malformed input on 2026-11-03; the receipt loader, the $\varepsilon$-checker's correction reader and the second verifier were fixed that day, every class became a standing vector, and every artifact the fix touched was re-run: the strict replay of every released receipt, the IBM re-certification, the semantics audit of Table I, the mutation score, and the producer-identity comparison. The preregistered gap run had been judged by the checker as committed before the fix; re-judged by the fixed checker it gives the same verdict, slack and bound on every shot.

*The replay's attacks, in detail.* The parity adversary builds, on each attacked receipt, a strictly heavier correction "proven" by sets that are not $T$-odd, its packing scaled so that the bound lands within $\varepsilon_{\max}$ below the heavier weight; whether such a forgery is real is decided by running it through copies of both verifiers with only their parity test removed. Unscaled, as the search first returns it, the packing's bound exceeds the heavier weight, and a parity-blind checker reports that as impossible rather than accepting it. On a sample of replayed receipts in the test suite a parity-blind copy of the JavaScript checker accepts the scaled forgeries too and the real one refuses them. The boundary attack raises one correction edge's weight to the smallest double that puts the proven slack past $\varepsilon_{\max}$, which both verifiers must refuse, or by half the remaining room, which both must accept; the substitution attack replays a receipt against another configuration's graph, where none may be accepted.

*A substituted graph that was the same graph.* The replay's substitution attack re-decides a receipt against another configuration's graph, where nothing may be accepted, and took the first other configuration by name. Once the fresh-seed rerun of the simulation was added, that was the simulation's own circuit, and the strict replay of the simulation reported every substituted receipt accepted. The attack now takes the first configuration whose graph differs, and records which; every other configuration's choice was already a different graph and is unchanged.

*The forgery battery's heavier classes.* Before 2026-11-03 the three classes meant to reach the bound appended one edge twice to the genuine correction; the battery's own falsity audit summed the doubled weight and called the claim false. Read as an edge set the claim was true, and the $\varepsilon$-checker refused the documents as malformed input (Sec. VI). The audit now refuses such an attack, and a mutant that removes that refusal is killed by the test suite.

*The seeded faults' doubled edge.* Until 2026-11-07 the stealth mutant now built by adding a disjoint cycle appended one unused edge twice. As an edge set, the object a correction is (Sec. II), that is the decoder's own correction; the experiment weighed the list, so every such case was counted defective and proven suboptimal, and the released corpus labelled them defective, the construction the forgery battery's stealth classes had until 2026-11-03 (Sec. VI). The mutation code now returns each mutant as the edge set it denotes and discards one equal to the decoder's correction, the detection harness refuses a correction that lists an edge twice, and both experiments and the corpus were re-run. The rule also showed that the equal-weight control had never produced an alternative: every case it gave was the decoder's own correction, which already sent each fired detector to the boundary. The superseded records are in the development repository's history.

*The original simulation run.* The tolerance-rule simulation lost 150 of its worker chunks of 250 shots (37,500 shots) to memory exhaustion on a shared workstation and was completed from 3 supplements with disjoint seeds, which themselves lost 99 chunks of 250 shots and 16 of 750 (36,750 shots; 265 chunks and 74,250 shots in all). Some of those deaths were allocation failures during certification, whose memory use depends on the syndrome, so its rate cannot exclude selection by difficulty; the rerun has no such gap.

*Development attacks that were not attacks.* Twice during development an "attack" was accepted and the checker was right: once the forged correction was an equal-weight alternative optimum, and once a construction meant to hide a negative value degenerated into a valid certificate for an optimal correction. Every forgery other than the non-finite ones is therefore re-verified to be false, independently of the checker, before its verdict is counted, and a run in which any forgery fails that test is void.

*The two open distance-7 shots.* Generation 6 left one shot per basis open. We located both in the chunk records; an exact-arithmetic oracle, whose answer is re-verified, finds PyMatching's correction exactly optimal on each (91 and 127 detectors fired), and the current exact tier stops on both at its cap of 200 separation rounds without converging. Generation 6 cost 3.87 s (X) and 3.48 s (Z) of core time per shot.

*The held-out distance-7 deviation.* The deviation that cycles through the six arms was driven by chunk run times; the largest accepted $\varepsilon$ and the battery's falsity count of two chunks had been displayed, as its file records. It also adds what the preregistration does not have: dispatch markers and a watchdog that enforce the stopping rule; the per-arm report in three views of Sec. VI; and the preregistered tests computed only between arms that both have completed chunks.

*The held-out distance-7 stop.* The run started on 2026-11-03 at 14:35:29 from a snapshot of commit `2008c81f`, arm after arm as committed. At 16:18:10, 5.13 core-hours in, it was stopped for the blind order amendment (Sec. VI), losing the three chunks then in flight, and restarted at 16:29 under the deviation's driver (commit `4030f88d`) with the deadline unchanged. The watchdog stopped it at 03:55:29 on 2026-11-04, the 40 core-hours of the original start, with 3 chunks being computed; nobody read a not-proven count before the stop record was written. Its 42 dispatch markers are the 46 chunks dispatched over the whole run less the 4 completed before the deviation, which the committed runner dispatched without markers; a supervisor that would have restarted the driver had it died (`heldout_d7_babysitter.py`) never had to act. The completed chunks were copied out of the snapshot and verified byte for byte, replayed in a partial-plan mode added to the replay tool for the purpose, and reported by `heldout_d7_partial.py`, whose record pins the producer's modules to `2008c81f` and the driver to `4030f88d`. On 2026-11-06 the report was re-run on the same copied sample to add Fisher's exact test beside the preregistered one; compared field by field with the earlier record, nothing else changed but the report's own commit and digest (`docs/PREREG_heldout_d7_2026-11-03_DEVIATIONS.md`, note 1).

*The gap experiment's scorer.* The first version of the gap experiment also read the separator of a decomposed error as a detector, producing phantom three-detector mechanisms; that was found and fixed before publication. The duplicate-count defect of Sec. VIII was found by an external audit on 2026-11-03. The preregistered run was started from commit `2008c81f`; its record names `70b3c4d8`, the commit at which it finished after other work had been committed, and the four files it is judged by are byte-identical at the two. The runner now records the commit when it starts and refuses a run whose judged files change while it runs.


[1] J. Edmonds and E. L. Johnson, Matching, Euler tours and the Chinese postman, Mathematical Programming **5**, 88 (1973).

[2] R. M. McConnell, K. Mehlhorn, S. Näher, and P. Schweitzer, Certifying algorithms, Computer Science Review **5**, 119 (2011).

[3] M. Blum and S. Kannan, Designing programs that check their work, Journal of the ACM **42**, 269 (1995).

[4] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, Topological quantum memory, Journal of Mathematical Physics **43**, 4452 (2002), arXiv:quant-ph/0110143.

[5] O. Higgott and C. Gidney, Sparse Blossom: correcting a million errors per core second with minimum-weight matching, Quantum **9**, 1600 (2025), arXiv:2303.15933.

[6] M. X. Goemans and D. P. Williamson, A general approximation technique for constrained forest problems, SIAM Journal on Computing **24**, 296 (1995).

[7] J. Edmonds, Paths, trees, and flowers, Canadian Journal of Mathematics **17**, 449 (1965).

[8] M. W. Padberg and M. R. Rao, Odd minimum cut-sets and b-matchings, Mathematics of Operations Research **7**, 67 (1982).

[9] C. Gidney, Stim: a fast stabilizer circuit simulator, Quantum **5**, 497 (2021), arXiv:2103.02202.

[10] Google Quantum AI and Collaborators, Quantum error correction below the surface code threshold, Nature **638**, 920 (2025), arXiv:2408.13687.

[11] Google Quantum AI, Data for "Quantum error correction below the surface code threshold" (2024), doi:10.5281/zenodo.13273331.

[12] A. A. Hagberg, D. A. Schult, and P. J. Swart, Exploring network structure, dynamics, and function using NetworkX, in *Proceedings of the 7th Python in Science Conference* (2008) pp. 11–15.

[13] S. T. Spitz, B. Tarasinski, C. W. J. Beenakker, and T. E. O'Brien, Adaptive weight estimator for quantum error correction in a time-dependent environment, Advanced Quantum Technologies **1**, 1800012 (2018).

[14] Google Quantum AI, Exponential suppression of bit or phase errors with cyclic error correction, Nature **595**, 383 (2021).

[15] R. Krishnamoorthy, F. Gerhardt, J. Knaute, T. Klir, S. R. Maschek, E. Schulze, T. Maras, A. Dotterweich, L. K. Lee, C. Bauckhage, and N. Piatkowski, Certified decoding of quantum LDPC codes (2026), arXiv:2608.25545.

[16] E. Takou, C. Benito, A. Vezvaee, D. A. Lidar, and K. R. Brown, Logical error estimation from syndrome data of surface-code experiments (2026), arXiv:2606.11496.

[17] S. Stein, S. Kan, C. Liu, A. Harkness, S. Garner, Z. Du, Y. Ding, Y. Mao, and A. Li, Calibration-conditioned FiLM decoders for low-latency decoding of quantum error correction evaluated on IBM repetition-code experiments (2026), arXiv:2601.16123.

[18] P. Panteleev and G. Kalachev, Degenerate quantum LDPC codes with good finite length performance, Quantum **5**, 585 (2021), arXiv:1904.02703.

[19] J. Roffe, D. R. White, S. Burton, and E. Campbell, Decoding across the quantum low-density parity-check code landscape, Physical Review Research **2**, 043423 (2020), arXiv:2005.07016.

[20] J. Roffe, LDPC: Python tools for low density parity check codes, `https://pypi.org/project/ldpc/` (2022), the citation its README requests; version 2.4.1, released 2025-12-08, used here.

[21] O. Higgott, T. C. Bohdanowicz, A. Kubica, S. T. Flammia, and E. T. Campbell, Improved decoding of circuit noise and fragile boundaries of tailored surface codes, Physical Review X **13**, 031007 (2023), arXiv:2203.04948.

[22] J. Edmonds, Maximum matching and a polyhedron with 0,1-vertices, Journal of Research of the National Bureau of Standards Section B **69B**, 125 (1965).

[23] A. Neumaier and O. Shcherbina, Safe bounds in linear and mixed-integer linear programming, Mathematical Programming **99**, 283 (2004).

[24] D. E. Steffy and K. Wolter, Valid linear programming bounds for exact mixed-integer programming, INFORMS Journal on Computing **25**, 271 (2013).

[25] K. K. H. Cheung, A. Gleixner, and D. E. Steffy, Verifying integer programming results, in *Integer Programming and Combinatorial Optimization*, Lecture Notes in Computer Science, Vol. 10328 (2017) pp. 148–160.

[26] L. Eifler and A. Gleixner, A computational status update for exact rational mixed integer programming, Mathematical Programming **197**, 793 (2023).

[27] Y. Wu, B. Li, K. Chang, S. Puri, and L. Zhong, Minimum-weight parity factor decoder for quantum error correction (2025), arXiv:2508.04969.

[28] V. Kolmogorov, Blossom V: a new implementation of a minimum cost perfect matching algorithm, Mathematical Programming Computation **1**, 43 (2009).

[29] J. Feldman, M. J. Wainwright, and D. R. Karger, Using linear programming to decode binary linear codes, IEEE Transactions on Information Theory **51**, 954 (2005).

[30] J. X. Li and P. O. Vontobel, LP decoding of quantum stabilizer codes, in *2018 IEEE International Symposium on Information Theory (ISIT)* (2018) pp. 1306–1310.

[31] J. Shen and H. Zhong, Capability-gated conformance testing of quantum error-correction decoder libraries (2026), arXiv:2609.07035.

[32] M. Ehatamm, Y. Lee, X. Wu, and R. Tao, End-to-end formalization of quantum error correction (2026), arXiv:2605.16523.